

# National University of Computer and Emerging Sciences

## Islamabad Campus

| Q # | ANS | Q # | ANS |
|-----|-----|-----|-----|
| 1   | B   | 26  | D   |
| 2   | D   | 27  | B   |
| 3   | C   | 28  | B   |
| 4   | C   | 29  | B   |
| 5   | B   | 30  | B   |
| 6   | B   | 31  | D   |
| 7   | C   | 32  | C   |
| 8   | B   | 33  | B   |
| 9   | B   | 34  | C   |
| 10  | C   | 35  | B   |
| 11  | D   | 36  | C   |
| 12  | C   | 37  | B   |
| 13  | C   | 38  | C   |
| 14  | C   | 39  | C   |
| 15  | A   | 40  | C   |
| 16  | C   | 41  | A   |
| 17  | B   | 42  | B   |
| 18  | C   | 43  | C   |
| 19  | B   | 44  | A   |
| 20  | A   | 45  | B   |
| 21  | D   | 46  | C   |
| 22  | D   | 47  | B   |
| 23  | B   | 48  | B   |
| 24  | B   | 49  | B   |
| 25  | A   | 50  | A   |

National University of Computer and Emerging Sciences  
Islamabad Campus

**Computer Architecture (EE3009)**

**Course Instructor(s):**

Ms. Noor ul Ain, Mr. Shams Farooq, Mr. Aqib Rehman

**Section(s): (CS All Sections)**

**Final Exam-Part B**

**Total Time (Hrs):** 3

**Total Marks:** 120

**Total Questions:** 8

**Date:** Jun 8, 2026

Roll No

Course Section

Student Signature

**Important Information:** The use or possession of mobile phones and other electronic devices during the exam is strictly prohibited, whether the device is powered on or off. Do not bring mobile phones, smartwatches, tablets, or other electronic gadgets into the examination hall. If you are found with any prohibited devices, your exam may be cancelled, and further disciplinary action may be taken.

Do not write below this line.

**Attempt all the questions.**

1. You can start PART B after returning PART A
2. Verify that you have Nine **(9) printed page** of the question paper including this page.
3. **Using Pencil is prohibited. Any part done with the pencil will not be marked and cannot be claimed for rechecking.** Write **neatly, concisely**. Overwriting and cutting in the final answer will result in a zero score and cannot be claimed for rechecking.

**4. Solve Question 1 – Question 4 on answer book and remaining questions on the question paper in provided space.**

|                | Q-1 | Q-2 | Q-3 | Q-4 | Q-5 | Q-6 | Q-7 | Q-8 | Total |
|----------------|-----|-----|-----|-----|-----|-----|-----|-----|-------|
| Marks Obtained |     |     |     |     |     |     |     |     |       |
| Total Marks    | 18  | 20  | 15  | 07  | 15  | 13  | 25  | 07  | 120   |

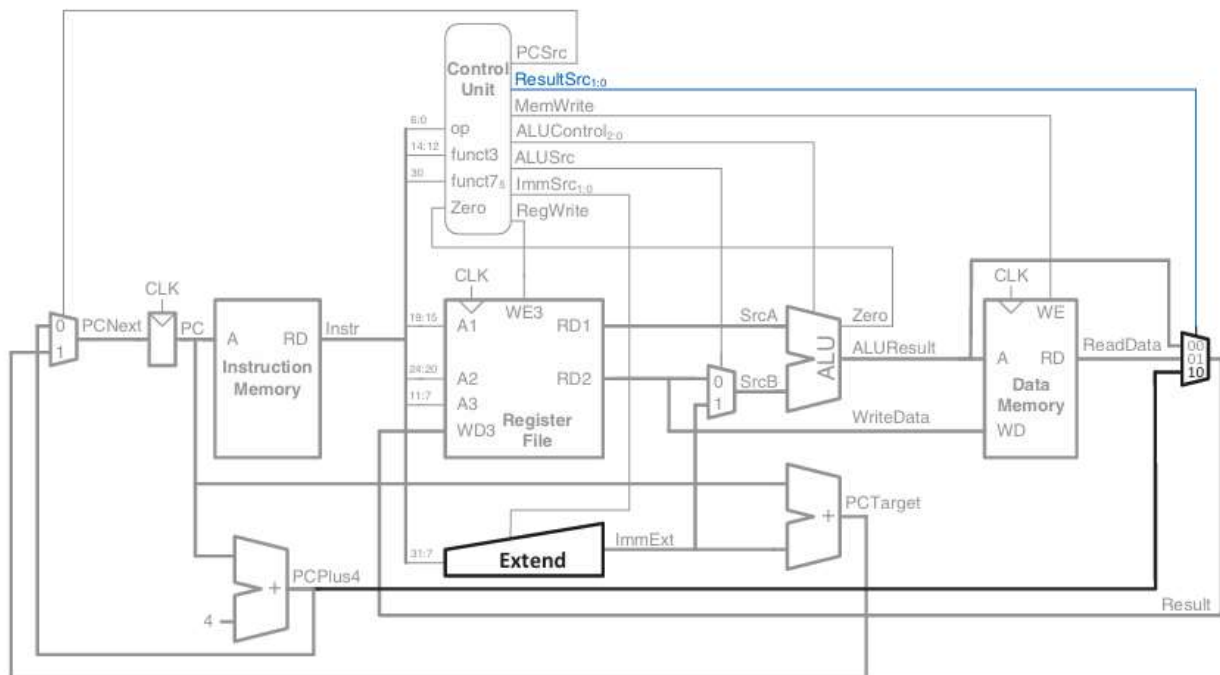
# National University of Computer and Emerging Sciences Islamabad Campus

## CHEAT Sheet

| Instruction | op            | funct3  | Funct7       | Type   |
|-------------|---------------|---------|--------------|--------|
| <b>add</b>  | 0110011 (51)  | 000 (0) | 0000000 (0)  | R-Type |
| <b>sub</b>  | 0110011 (51)  | 000 (0) | 0100000 (32) | R-Type |
| <b>and</b>  | 0110011 (51)  | 111 (7) | 0000000 (0)  | R-Type |
| <b>or</b>   | 0110011 (51)  | 110 (6) | 0000000 (0)  | R-Type |
| <b>addi</b> | 0010011 (19)  | 000 (0) | -            | I-Type |
| <b>beq</b>  | 1100011 (99)  | 000 (0) | -            | B-Type |
| <b>bne</b>  | 1100011 (99)  | 001 (1) | -            | B-Type |
| <b>lw</b>   | 0000011 (3)   | 010 (2) | -            | I-Type |
| <b>sw</b>   | 0100011 (35)  | 010 (2) | -            | S-Type |
| <b>jal</b>  | 1101111 (111) | -       | -            | J-Type |
| <b>jalr</b> | 1100111 (103) | 000 (0) | -            | I-Type |
| <b>lui</b>  | 0110111 (55)  | -       | -            | U-Type |

| Name           | Register Number | Usage                              |
|----------------|-----------------|------------------------------------|
| <b>zero</b>    | x0              | Constant value 0                   |
| <b>ra</b>      | x1              | Return address                     |
| <b>sp</b>      | x2              | Stack pointer                      |
| <b>gp</b>      | x3              | Global pointer                     |
| <b>tp</b>      | x4              | Thread pointer                     |
| <b>t0-2</b>    | x5-7            | Temporaries                        |
| <b>s0 / fp</b> | x8              | Saved register / Frame pointer     |
| <b>s1</b>      | x9              | Saved register                     |
| <b>a0-1</b>    | x10-11          | Function arguments / return values |
| <b>a2-7</b>    | x12-17          | Function arguments                 |
| <b>s2-11</b>   | x18-27          | Saved registers                    |
| <b>t3-6</b>    | x28-31          | Temporaries                        |

|                                 |        |        |        |                       |        |        |
|---------------------------------|--------|--------|--------|-----------------------|--------|--------|
| 7 bits                          | 5 bits | 5 bits | 3 bits | 5 bits                | 7 bits |        |
| funct7                          | rs2    | rs1    | funct3 | rd                    | op     | R-Type |
| imm <sub>11,0</sub>             | rs1    | funct3 | rd     | op                    |        | I-Type |
| imm <sub>11,5</sub>             | rs2    | rs1    | funct3 | imm <sub>4,0</sub>    | op     | S-Type |
| imm <sub>12,10:5</sub>          | rs2    | rs1    | funct3 | imm <sub>4,1,11</sub> | op     | B-Type |
| imm <sub>31:12</sub>            |        |        | rd     | op                    |        | U-Type |
| imm <sub>20,10:1,11,19:12</sub> |        |        | rd     | op                    |        | J-Type |
| 20 bits                         |        |        | 5 bits | 7 bits                |        |        |



# National University of Computer and Emerging Sciences

## Islamabad Campus

**Q1:** Recently company “CA Tigers” hired some new designers at low salaries in order to save the cost, but it results in with **big disaster**. During the manufacturing due to some **designing glitch** some control signals are becomes **faulty** in the single-cycle RISC-V processor. Some control signals **stuck-at-0 fault**, meaning that the signal is always 0 regardless of the value generated by the control unit. For **each of the following control signals**, analyze the impact of the fault, your answer must include the following:

- Which RISC-V instruction types (lw, sw, or, beq, jal, addi) would malfunction?
- Describe how the behavior of the affected instructions would change.
- Explain why the fault causes the malfunction by referring to the datapath and control logic.

### Part A [3\*3 =9 Marks]

(i) **Control Signal – RegWrite**

Affected/Malfunctioned Instructions

Justification of your answer

#### Affected/Malfunctioned Instructions

lw  
or  
addi  
jal

#### Instructions that still work

sw  
beq

#### Justification

RegWrite enables writing into the Register File.

When stuck at 0:

Register File Write Enable = 0

No register can be updated.

(ii) **Control Signal – MemWrite**

Affected/Malfunctioned Instructions

Justification of your answer

#### Affected/Malfunctioned Instructions

sw

#### Instructions that still work

lw  
or  
addi  
beq  
jal

#### Justification

MemWrite enables writing into Data Memory.

When stuck at 0:

Memory writes never occur.

# National University of Computer and Emerging Sciences

## Islamabad Campus

(iii) **Control Signal – ALUSrc**

Affected/Malfunctioned Instructions

Justification of your answer

### Affected/Malfunctioned Instructions

lw  
sw  
addi

### Instructions that still work

or  
beq  
jal

### Justification

ALUSrc selects ALU's second input.

When stuck at 0:

ALU always receives rs2  
instead of immediate.

**Part B** Repeat Exercise Q1 Part A, assuming that the signal has a **stuck-at-1** fault. [3\*3 =9 Marks]

(i) **Control Signal – PCSrc**

Affected/Malfunctioned Instructions

Justification of your answer

### Affected/Malfunctioned Instructions

All instructions

### Justification

If permanently 1:

Processor always selects branch target path.

(ii) **Control Signal – ResSrc1**

Affected/Malfunctioned Instructions

Justification of your answer

### Affected/Malfunctioned Instructions

or  
addi  
jal

### Instructions that still work

lw  
sw  
beq

### Justification

When stuck at 1:

Write-back data always comes from Data Memory.

# National University of Computer and Emerging Sciences

## Islamabad Campus

### (iii) Control Signal – ALUSrc

Affected/Malfunctioned Instructions

Justification of your answer

#### Affected/Malfunctioned Instructions

or  
beq

#### Instructions that still work

lw  
sw  
addi

#### Justification

When stuck at 1:

ALU second operand always comes from Immediate Generator.

**Q2:** To improve the **performance** of programs that **frequently access arrays**, the RISC-V ISA is to be extended with a new instruction called **lwpostinc** (Load Word with Post-Increment). The Instruction **lwpostinc rd, imm(rs)** has the same effect as executing the following two instructions:

lw rd, 0(rs)

addi rs, rs, imm

#### **Part A [2 Marks]**

Assume rd = S3, imm = 4, rs = s1 and the following initial values: s1 = 1000, Memory[1000] = 25

Show the final values of: S1 and S3 Registers

s3 = 25

s1 = 1004

#### **Part B [3\*3=9 Marks]**

Using the single-cycle RISC-V datapath, identify:

- (i) Which existing components can be reused and why?

#### Instruction Memory

Used to fetch lwpostinc.

#### Register File

Used to read:

rs = s1

#### ALU

Can compute:

Address = rs + 0

for load operation.

Also can compute:

rs + imm

# National University of Computer and Emerging Sciences

## Islamabad Campus

### Data Memory

Used to read:

Memory[rs]

### Immediate Generator

Already generates:

imm

- (ii) Which components would need to be modified and added?

#### 1. Register File Modification

##### Problem:

Two registers must be updated simultaneously:

$rd \leftarrow \text{loaded value}$

$rs \leftarrow \text{incremented address}$

#### 2. Additional Adder / ALU Path

The processor must compute:

$\text{Address} = rs + 0$  (for lw)

and

$rs + \text{imm}$  (for post increment)

at the same time.

- (iii) Why the current datapaths cannot execute lwpostinc in a single cycle.

This requires:

##### Write #1

$rd \leftarrow \text{loaded value}$

##### Write #2

$rs \leftarrow \text{incremented address}$

The standard register file supports only:

ONE write per cycle

Therefore:

lwpostinc requires TWO register writes.

The existing datapath cannot do this.

### Part C [3+2=5 Marks]

Determine the values of the following control signals for lwpostinc:

|                      |           |        |
|----------------------|-----------|--------|
| RegWrite             | RegWrite  | 1      |
| ALUSrc               | ALUSrc    | 1      |
| MemWrite             | MemWrite  | 0      |
| ResultSrc (MemtoReg) | ResultSrc | Memory |
| Branch               | Branch    | 0      |
| ALUOp                | ALUOp     | ADD    |

If additional control signals are required, clearly specify them.

# National University of Computer and Emerging Sciences

## Islamabad Campus

Need a control signal such as:

PostIncEnable

which enables:

$rs \leftarrow rs + imm$

### **Part D [4 Marks]**

Explain why `lwpostinc` creates a problem for the existing register file design.

Standard register file:

1 write port

supports:

One destination register

One write data value

But `lwpostinc` needs:

Write Port #1

$rd \leftarrow Memory[rs]$

Write Port #2

$rs \leftarrow rs + imm$

simultaneously.

What modification would be required to support this instruction in a single cycle?

Add:

Second write port

to the register file.

New register file:

2 Read Ports

2 Write Ports

**Q3:** Consider the delays given in **Reference Table 3AB** at the end of this question. Your friend is a **crack circuit designer**. She has offered to **redesign some of the units** in the **single-cycle RISC-V processor** to have half the delay. Due to **budget issue** we can **only** afford redesigning of **two** units.

### **Part A [8+2 =10 Marks]**

- (i) Which units should she work on to obtain the greatest speedup of the overall processor, and what would the cycle time of the new improved machine be? Mandatory to show the rough work along the final answer's.

The critical path is the `lw` instruction:

$$T_{c\_single} = t_{pcq\_PC} + 2t_{mem} + t_{RFread} + t_{ALU} + t_{mux} + t_{RFsetup}$$

The cycle time is:

$$= 40 + 2(200) + 100 + 200 + 30 + 100 = 870 \text{ ps}$$

The units with the largest delay are Memory and ALU (200 ps each). Reducing them by half decreases the cycle time to:

$$870 - 300 = 570 \text{ ps}$$



# National University of Computer and Emerging Sciences

## Islamabad Campus

(ii) What would be the **speedup** of new improved machine be?

Redesigning **Memory, ALU** gives the greatest improvement. The new cycle time is **570 ps**, and the speedup is:  
 $870 / 570 \approx 1.52\times$

### **Part B [5 Marks]**

In the summer vacation your friend builds a **prefix adder** that reduces the ALU delay by **20 ps**. If the other element delays stay the same as given in the reference table 3AB, determine how long your **single-cycle RISC-V processor** takes to execute a benchmark with 50 billion instructions.

**Prefix adder reduces ALU delay by 20 ps:**

$$t_{\text{ALU, new}} = 200 - 20 = 180$$

$$T_{\text{c\_single}} = t_{\text{pcq\_PC}} + 2t_{\text{mem}} + t_{\text{RFread}} + t_{\text{ALU}} + t_{\text{mux}} + t_{\text{RFsetup}}$$

The cycle time is:

$$= 40 + 2(200) + 100 + 180 + 30 + 100 = 850 \text{ ps}$$

Program with 50 billion instructions:

$$\begin{aligned} \text{Execution Time} &= \# \text{ instructions} \times \text{CPI} \times T_{\text{c}} \\ &= (50 \times 10^9) (1) (850 \times 10^{-12} \text{ s}) \\ &= \mathbf{42.5 \text{ seconds}} \end{aligned}$$

**Reference Table 3AB — Delay of circuit elements**

| Element             | Parameter | Delay(ps) |
|---------------------|-----------|-----------|
| Register (clk-to-Q) | t_pcq_PC  | 40        |
| Multiplexer         | t_mux     | 30        |
| AND-OR gate         | t_AND-OR  | 20        |
| ALU                 | t_ALU     | 200       |
| Memory              | t_mem     | 200       |
| Register file read  | t_RFread  | 100       |
| Register file setup | t_RFsetup | 100       |

# National University of Computer and Emerging Sciences

## Islamabad Campus

**Q4:**

### **Part A**

A software company operates a large web server that processes thousands of user requests every minute. The engineers discover that a particular module responsible for handling encrypted connections can be upgraded. On the current server, processing a request through this module takes 100 ns. After the upgrade, the same operation takes only 20 ns. However, not every request uses this module. **[3+3 =6 Marks]**

- (i) Suppose only 30% of the total execution time of the server is spent in this module, while the remaining time is spent in other parts of the system that are unaffected by the upgrade. What is the overall speedup of the server after the enhancement?

$$\text{Overall Speedup} = \frac{1}{(1 - F) + \frac{F}{S}}$$

$$\text{Fraction}_{\text{enhanced}} = 30\% = 0.30$$

$$\text{Speedup}_{\text{enhanced}} = 100/20 = 5$$

$$\bullet \text{ Speedup}_{\text{overall}} = 1 / \{ (1-0.3) + 0.3/5 \} = 1.32$$

- (ii) After observing the results from part (A), the engineers propose a more optimization that would reduce the module's execution time from 100 ns to 10 ns instead of 20 ns. The new overall speedup?

$$\text{Fraction}_{\text{enhanced}} = 30\% = 0.30$$

$$\text{Speedup}_{\text{enhanced}} = 100/10 = 10$$

$$\bullet \text{ Speedup}_{\text{overall}} = 1 / \{ (1-0.3) + 0.3/10 \} = 1.37$$

### **Part B** **[1 Mark]**

Checkpoint Reached! Just like a pipelined processor, keep moving forward—every completed question brings you closer to the finish line. 😊

Write this line:

I will not give up; I can do this.

**I will not give up; I can do this.**

# National University of Computer and Emerging Sciences

## Islamabad Campus

### [CLO 3: Design and analyze single-cycle and pipelined microarchitectures for RISC-V ISA]

**Q5:** The pipelined RISC-V processor is running the following code snippet. Which registers are being written and which are being read on the fifth cycle? Recall that the pipelined RISC-V processor has a Hazard Unit. You may assume a memory system that returns the result within one cycle. Assume memory index and content is same for fetching the value from memory like memory[25] has 25 value. Show any bubbles if required to fill the following table till cycle #6. **[Marks 15=12+3]**

```

1. addi s1, zero, 52      # s1 = 52
2. addi s0, s1, -4        # s0 = s1 - 4 = 48
3. lw s3, 16(s0)          # s3 = memory[64]
4. sw s3, 20(s0)          # memory[68] = s3
5. xor s2, s0, s3         # s2 = s0 ^ s3
6. or s2, s2, s3          # s2 = s2 | s3

```

| Cycle | Fetch | Decode | Execute | Memory | Writeback |
|-------|-------|--------|---------|--------|-----------|
| 1     |       |        |         |        |           |
| 2     |       |        |         |        |           |
| 3     |       |        |         |        |           |
| 4     |       |        |         |        |           |
| 5     |       |        |         |        |           |
| 6     |       |        |         |        |           |

**Registers Values only at 5<sup>th</sup> Cycle:**

Bubble has two marks last row has more weightage

# National University of Computer and Emerging Sciences Islamabad Campus

## Exercise 7.31

The code executes as follows:

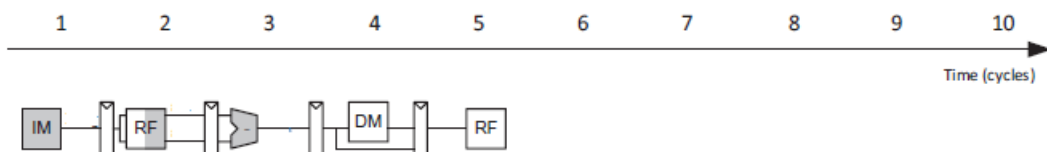
| Pipeline stages |               |               |               |               |               |
|-----------------|---------------|---------------|---------------|---------------|---------------|
| Cycle           | Fetch         | Decode        | Execute       | Memory        | Writeback     |
| 1               | addi s1,x0,52 |               |               |               |               |
| 2               | addi s0,s1,-4 | addi s1,x0,52 |               |               |               |
| 3               | lw s3,16(s0)  | addi s0,s1,-4 | addi s1,x0,52 |               |               |
| 4               | sw s3,20(s0)  | lw s3,16(s0)  | addi s0,s1,-4 | addi s1,x0,52 |               |
| 5               | xor s2,s0,s3  | sw s3,20(s0)  | lw s3,16(s0)  | addi s0,s1,-4 | addi s1,x0,52 |
| 6               | xor s2,s0,s3  | sw s3,20(s0)  | bubble        | lw s3,16(s0)  | addi s0,s1,-4 |

In cycle 5, **s1** is being written (by addi) in the Decode stage, and **s0** and **s3** are being read by sw in the Decode stage. Note that in this cycle, sw detects that it needs to stall in the next cycle – because a lw is in the Execute stage that will produce one of its source registers (s3), and the lw won't have that operand ready until the end of the Memory stage.

**Q6:** Using a diagram notation given below, show the forwarding and stalls needed to execute the instructions given below on the pipelined RISC-V processor. **[Marks 13]**

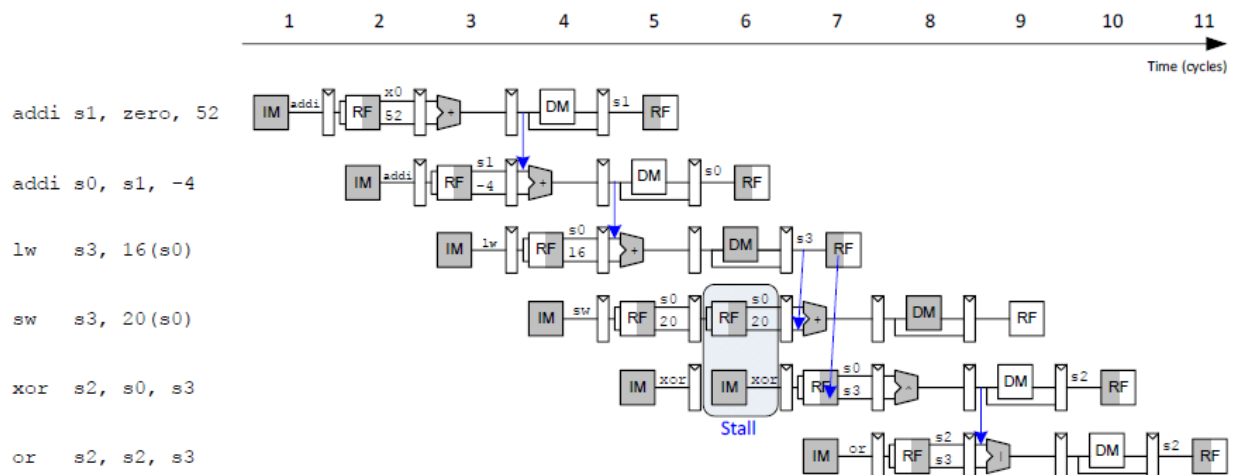
- |                       |                    |
|-----------------------|--------------------|
| I1. addi s1, zero, 52 | # s1 = 52          |
| I2. addi s0, s1, -4   | # s0 = s1 - 4 = 48 |
| I3. lw s3, 16(s0)     | # s3 = memory[64]  |
| I4. sw s3, 20(s0)     | # memory[68] = s3  |
| I5. xor s2, s0, s3    | # s2 = s0 ^ s3     |
| I6. or s2, s2, s3     | # s2 = s2   s3     |

**Use following diagram notation to draw solution**



# National University of Computer and Emerging Sciences Islamabad Campus

## Exercise 7.33



Stall=2 , forward arrows=5, 6 correct instructions

**Q7:** Consider the following instruction sequence in **RISC-V assembly language**: [Marks=25]

**Register values:** x8 = 0x10000000 (valid aligned memory address)      x9 = 0x00000000

x10 = any value      MEM[x8] = any valid word

```

I1: L3 : lw  x20, 0(x8)
I2:  add x21, x20, x9
I3:  xor x22, x20, x10
I4:  and x23, x21, x9
I5:  or  x8,  x22, x9
I6:  beq x23, x0, L3
    
```

**Part A — No Forwarding** (RF same-cycle read/write applies)

Write **IF / ID / EX / MEM / WB** in active cells. Write **s** for a stall bubble. Leave inactive cells blank. **Do NOT insert NOP instructions** — hardware stalling holds the pipeline register.

Note: cutting and overwriting the values in all the tables below will result in zero score

(i) Complete the timing diagram below. [Marks=06]

| Instruction           | CC1 | CC2 | CC3 | CC4 | CC5 | CC6 | CC7 | CC8 | CC9 | CC10 | CC11 | CC12 | CC13 | CC14 |
|-----------------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|------|------|------|
| I1: lw x20, 0(x8)     | IF  | ID  | EX  | MEM | WB  |     |     |     |     |      |      |      |      |      |
| I2: add x21, x20, x9  |     | IF  | S   | S   | ID  | EX  | MEM | WB  |     |      |      |      |      |      |
| I3: xor x22, x20, x10 |     |     |     |     | IF  | ID  | EX  | MEM | WB  |      |      |      |      |      |
| I4: and x23, x21, x9  |     |     |     |     |     | IF  | S   | ID  | EX  | MEM  | WB   |      |      |      |
| I5: or x8, x22, x9    |     |     |     |     |     |     |     | IF  | ID  | EX   | MEM  | WB   |      |      |

# National University of Computer and Emerging Sciences

## Islamabad Campus

| Instruction         | CC1 | CC2 | CC3 | CC4 | CC5 | CC6 | CC7 | CC8 | CC9 | CC10 | CC11 | CC12 | CC13 | CC14 |
|---------------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|------|------|------|
| I6: beq x23, x0, L3 |     |     |     |     |     |     |     |     | IF  | S    | ID   | EX   | MEM  | WB   |

(ii) Identify all RAW (Read after Write) data hazards — complete the table : [Marks=05]

For Row 5, also state how many cycles the branch flush adds.

| Producing Instruction | Consuming Instruction | Hazard Register | No. of Stall Cycles (write 0 if none) |
|-----------------------|-----------------------|-----------------|---------------------------------------|
| I1                    | I2                    | X20             | 2                                     |
| I1                    | I3                    | X20             | 0                                     |
| I2                    | I4                    | X21             | 1                                     |
| I3                    | I5                    | X22             | 0                                     |
| I4                    | I6: beq x23, x0, L3   | X23             | Flush 3 (1+2)                         |

Note: single wrong entry in a row will lead to zero score for that row

**Part B — Full Forwarding Enabled** (MEM→EX and WB→EX) + RF same-cycle read/write

Fill stages as in Part A. Also annotate each EX cell that receives a forwarded value — write the register and source, e.g. "EX x20-WB". Mark remaining stall bubbles with s and circle them.

(i) Complete the timing diagram below. [Marks=07]

| Instruction           | CC1 | CC2 | CC3 | CC4 | CC5 | CC6 | CC7 | CC8 | CC9 | CC10 | CC11 | CC12 | CC13 | CC14 |
|-----------------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|------|------|------|
| I1: lw x20, 0(x8)     | IF  | ID  | EX  | MEM | WB  |     |     |     |     |      |      |      |      |      |
| I2: add x21, x20, x9  |     | IF  | ID  | S   | EX  | MEM | WB  |     |     |      |      |      |      |      |
| I3: xor x22, x20, x10 |     |     | IF  | S   | ID  | EX  | MEM | WB  |     |      |      |      |      |      |
| I4: and x23, x21, x9  |     |     |     |     | IF  | ID  | EX  | MEM | WB  |      |      |      |      |      |
| I5: or x8, x22, x9    |     |     |     |     |     | IF  | ID  | EX  | MEM | WB   |      |      |      |      |
| I6: beq x23, x0, L3   |     |     |     |     |     |     | IF  | ID  | EX  | MEM  | WB   |      |      |      |

(ii) Total cycles for one iteration = \_\_\_\_11\_\_\_\_

[Marks=02]

(iii) Identify all remaining stalls and forwarding paths — complete the table: [Marks=05]

| Producing Instruction | Consuming Instruction | Hazard Register | Stall Cycles | Forwarding Path (describe, e.g. "WB→EX" or "RF same-cycle") |
|-----------------------|-----------------------|-----------------|--------------|-------------------------------------------------------------|
| I1                    | I2                    | X20             | 1            | WB→EX                                                       |
| I1                    | I3                    | X20             | 0            | RF same-cycle                                               |
| I2                    | I4                    | X21             | 0            | EX→ID                                                       |

# National University of Computer and Emerging Sciences

## Islamabad Campus

| Producing Instruction | Consuming Instruction | Hazard Register | Stall Cycles   | Forwarding Path<br>(describe, e.g. "WB→EX" or "RF same-cycle") |
|-----------------------|-----------------------|-----------------|----------------|----------------------------------------------------------------|
| I3                    | I5                    | X22             | 0              | I3                                                             |
| I4                    | I6: beq x23, x0, L3   | X23             | Flush 3<br>(2) | EX→ID                                                          |

*Note: single wrong entry in a row will lead to zero score for that row*

**Q8: Consider the following reference Table 7.7**

**Reference: Table 7.7 — Logic Element Delays (provided for all questions)**

| Element             | Parameter | Delay (ps) |
|---------------------|-----------|------------|
| Register (clk-to-Q) | t_pcq     | 40         |
| Register setup      | t_setup   | 50         |
| Multiplexer         | t_mux     | 30         |
| AND-OR gate         | t_AND-OR  | 20         |
| ALU                 | t_ALU     | 120        |
| Memory read         | t_mem     | 200        |
| Register file read  | t_RFread  | 100        |
| Register file setup | t_RFsetup | 60         |

# National University of Computer and Emerging Sciences

## Islamabad Campus

- a) Write the critical path formula for the single-cycle processor cycle time. **[Marks 02]**
- b) A new memory chip reduces  $t_{\text{mem}}$  by 30%. Calculate the new Fetch and Memory stage delays. Does the cycle time change? **[Marks 02]**
- c) An engineer claims: 'Doubling the speed of the register file read will halve the overall cycle time.' Is this claim correct? Prove your answer numerically. **[Marks 02]**
- d) Which single component from Table 7.7, if made infinitely fast (0 ps), gives the greatest reduction in cycle time? What would the new cycle time be? **[Marks 02]**

a)  $T_c = t_{\text{pcq}} + 2 \cdot t_{\text{mem}} + t_{\text{RFread}} + t_{\text{ALU}} + t_{\text{mux}} + t_{\text{RFsetup}}$   
(The lw instruction defines the critical path: IF → RF read → ALU → Dmem → RF write)

### MODEL ANSWER

- a) New  $t_{\text{mem}} = 200 \times 0.70 = 140$  ps  
New Fetch =  $40 + 140 + 50 = 230$  ps (was 290 ps)  
New Memory =  $40 + 140 + 50 = 230$  ps (was 290 ps)  
All other stages unchanged: Decode=300 ps, Execute=350 ps, Writeback=260 ps  
 $T_c = \max(230, 300, 350, 230, 260) = 350$  ps — NO CHANGE.  
Reason: Execute is still the bottleneck. Improving memory only helps Fetch/Memory, which are not on the critical path.
- b) Claim is FALSE. Current Decode =  $2(t_{\text{RFread}} + t_{\text{setup}}) = 2(100+50) = 300$  ps.  
With  $t_{\text{RFread}} = 0$ : Decode =  $2(0+50) = 100$  ps.  
 $T_c = \max(290, 100, 350, 290, 260) = 350$  ps — still 350 ps!  
Execute is the bottleneck, so improving the register file read has NO effect on  $T_c$ .
- c) The ALU ( $t_{\text{ALU}} = 120$  ps) is in the Execute stage (the bottleneck). Removing it:  
Execute =  $40 + 4(30) + 0 + 20 + 50 = 230$  ps  
 $T_c = \max(290, 300, 230, 290, 260) = 300$  ps (Decode becomes bottleneck)  
Greatest single reduction: ALU → new  $T_c = 300$  ps (reduction of 50 ps).